INGRES-Text2SQL

Division of work, methods, and viva preparation

Multilingual (English / Hindi / Hinglish / Tamil) natural-language interface to India's official groundwater database (INGRES, Central Ground Water Board + IIT Hyderabad). Repository: github.com/WAkshat/NLP-INGRES. Team: Akshat, Aryan, Akshit, Waie.

1. The project in one page

Problem. INGRES publishes groundwater assessments for about 7,000 blocks/taluks/mandals per cycle (recharge, extraction, stage of extraction, and a category: Safe / Semi-Critical / Critical / Over-Exploited). The portal is an English-only, click-through map with no public download and no documented API. The people who need the data (farmers, district officials, journalists) often ask in Hindi, Hinglish or regional languages, e.g. *"Ludhiana ke kitne blocks over-exploited hain?"*

Goal. Translate such questions into correct SQL over the INGRES data, run it safely, and answer with numbers that are checked against the data, and **measure which components actually improve accuracy**.

Why it is hard. (1) Three scripts (Latin, Devanagari, Tamil) and code-mixing, while the schema is in English; (2) 8,751 places, 169 unit names shared across states (e.g. 'Ramnagar' x10), idiosyncratic spellings ('TAMILNADU'); (3) domain conventions ('the 2025 assessment' = assessment_year '2024-2025'; volumes in hectare-metres); (4) unit IDs change between cycles; (5) a fluent but wrong number is dangerous for official data.

Main metric. Execution accuracy (EX): a question counts as correct if the predicted SQL returns the same result as the gold SQL.

Result	Value
Database vs government-published national totals	match within 0.07% (4 cycles)
Benchmark built (INGRES-Bench v1)	371 questions x 4 languages = 1,484 instances, 0 QC failures
Keyword/template baseline EX	38.8% (test split 32.6%)
Local LLM qwen3:8b zero-shot EX	18.2% (test 18.0%)
Schema linker column recall@3	0.409 -> 0.896 (fine-tuned); 0.851 on unseen question types
Place resolver, ambiguous names	0.447 -> 0.973 top-1 with state context
LLM + schema-linker hints (test)	23.2% EX vs 18.0% (+5.2 points, McNemar p = 0.008)
Unsupported numbers in answers	0.83% -> 0% with the numeric verifier
Full pipeline EX (test / hard_test)	**71.9% / 57.7%** (baselines 32.6% / 30.8% and 18.0% / 13.1%; p < 1e-6)
Biggest contributors (test, leave-one-out)	retrieved examples +36.5, place hints +4.4, repair +1.6

2. Division of work

Member	Role	Phases / components
Akshat	Data engineering, system integration, project lead	Ph 1-2 data access + database, SQL safety layer, Ph 9 full pipeline (71.9% EX), Ph 11 ablations, error analysis, docs
Aryan	Benchmark, evaluation, baselines	Ph 3 INGRES-Bench, execution-accuracy evaluator, Ph 4 baselines, LLM clients and prompts
Akshit	Learned models	Ph 5 schema linker, Ph 6 entity resolver, place linker
Waie	Multilingual analysis, answer trust, UI	Ph 7 tokenization study, Ph 8 numeric verifier, Ph 10 UI, mid-term report and slides

How the pieces connect at run time: question -> schema linker (Akshit) suggests columns -> place linker (Akshit) gives exact database spellings -> similar training questions retrieved (Aryan's benchmark) -> prompt (Aryan) -> qwen3:8b on the RTX 5060 writes SQL -> read-only executor (Akshat) runs it, errors are fed back for repair (Akshat) -> answer sentence -> numeric verifier (Waie) -> UI (Waie). Every step is scored by the evaluator (Aryan) on the benchmark (Aryan).

3. Akshat: data engineering, integration, project lead

What was built

- **Data access (Phase 1).** Found INGRES's undocumented public endpoint `POST /api/gec/getBusinessDataForUserOpen` and crawled 7 assessment cycles: 5,299 requests, 0 failures, ~80 min.
- **Canonical database (Phase 2).** Flattened 6.2 million raw facts into a 9-table SQLite database (21.8 MB): 5 cycles, 37 states, 770 districts, 8,751 units, 34,565 unit-year rows.
- **Cross-year crosswalk.** Links the same unit across cycles even when INGRES re-issued its ID.
- **Read-only SQL safety layer** used by every component.
- **Full pipeline (Phase 9)** that joins all components, with an execution-guided repair loop; final evaluation.

How and why

Finding the data

How: the portal's HTML is an empty Angular shell, so the data had to come from a background API. Reading the portal's JavaScript bundle revealed the base URL and a function used for non-logged-in users that posts to the endpoint above without authentication. Opening a location returns its children, so the crawler walks country -> state -> district, sends one request at a time with a 0.5 s delay, and caches every raw response (gzip, with the request and timestamp) so the crawl is resumable and reproducible.

Why: Excel export is admin-only and PDF reports would need error-prone table extraction. The JSON API gives the same numbers in structured form. Politeness (sequential requests, delay) avoids loading a government server.

Proving the data is right

How: national totals computed from our database were compared with the figures the government published in PIB press releases. Official figures count only fresh groundwater, so fresh = total - saline (poor_quality). Result: GWRA 2022 recharge 437.61 vs 437.60 BCM, extraction 239.18 vs 239.16; GWRA 2025 448.51 vs 448.52 and 247.22 vs 247.22; largest deviation 0.07%. This is an automated test (`tests/test_db_against_published.py`).

Why: without this, any later accuracy number could be measuring a broken database. It also caught a real bug: the API omits the saline key when it is zero; storing it as NULL silently dropped rows and gave a wrong national extraction of 128 BCM. The database now stores 0 in that case.

Two cycles were excluded on evidence: 2016-17 (4 states missing, totals about 2x the published values) and 2025-26 (incomplete, cycle in progress).

Cross-year crosswalk

How: walking backwards from 2024-25, each older unit is matched in order: (1) same ID; (2) same name in the same district; (3) name unique in the state; (4) fuzzy match within the same district, accepted only if it is the mutual best match with similarity >= 0.75 and not a split unit (SANGANER vs SANGANER_RURAL) or a different ordinal (-II vs -III). Ambiguous candidates are never linked. A hand check of 40 fuzzy links: 37 correct (92.5%).

Why: unit-ID overlap between 2019-20 and 2021-22 is only 0.14, so naive joins across years are wrong. States where the unit type changed (e.g. Tamil Nadu firka -> taluk) are recorded in a table and excluded from cross-year questions, because those units are not comparable.

SQL safety

How: the database is opened read-only, and an SQLite authorizer callback allows only SELECT/READ/function actions, blocks dangerous functions (load_extension, readfile, writefile), enforces a 10 s timeout and a row cap. 13 tests try injections, writes and multiple statements. **Why:** the SQL is written by an LLM from user text, so it must be treated as untrusted. Read-only access is enforced by the database engine, not by checking the SQL

string.

Full pipeline and repair loop

How: `Pipeline.sql` builds one prompt from the schema-linker hints, the resolved places and 3 retrieved examples; if the SQL errors or returns nothing, the error message is sent back to the model (at most 2 repairs; it stops if the model repeats the same query). Each component can be switched off for ablation. **Why:** zero-shot errors were systematic (wrong year label, wrong column, wrong place spelling), and each component targets one of them; ablations show which ones actually help.

Final results and ablations (Phases 9 and 11)

System	test EX (n=384)	hard_test EX (n=260)
Keyword/template baseline	32.6%	30.8%
qwen3:8b zero-shot	18.0%	13.1%
+ schema hints	23.2%	-
+ place hints + repair (no examples)	35.4%	-
Full pipeline (+ 3 retrieved examples)	**71.9%** [67.4, 76.0]	**57.7%** [51.9, 63.8]
Full without place hints	67.4% (-4.4, p = 0.019)	-
Full, first attempt only (no repair)	70.3% (-1.6; 6 fixed, 0 broken)	56.1%

By language (test): English 78.1, Hindi 69.8, Hinglish 77.1, Tamil 62.5. Error analysis of 218 failures: place not linked 41%, other logic 33%, wrong year 13%, wrong column 8%, wrong aggregation 2%, execution error 2%; place errors are worst in Tamil. **Caveat:** 376 of 384 test questions use question types that also appear in training (with other places, years and wording), so retrieved examples benefit from the template structure; the keyword baseline has the same advantage and is still beaten by 39 points.

Main files to show

File	What to point at
`src/ingestion/ingres_api.py`	IngresClient.fetch (line 48): one cached request; crawl_year (85): the country->state->district walk
`src/ingestion/flatten.py, build_db.py`	flatten (44): nested JSON to long table; build (101): creates the 9 SQLite tables
`src/normalization/crosswalk.py`	crosswalk (72): the 4-stage matcher; _mutual_best (56): guarded fuzzy matching
`src/utils/safe_sql.py`	_authorizer (36): allow-list; execute (50): timeout + row cap
`src/pipeline/agent.py`	Pipeline.sql (122): prompt + repair loop; Pipeline.answer (151)
`docs/DATA_ACCESS_REPORT.md`	the full data audit and reconciliation table

Viva questions and answers

Q. How did you get the data if there is no API?

The portal is a JavaScript app. Its bundle contains the API base URL and the request the site itself makes for non-logged-in visitors. We call that same public endpoint with plain HTTPS POST requests, slowly and sequentially, and keep every raw response so the dataset can be rebuilt without re-crawling.

Q. How do you know your database is correct?

We recompute national totals and compare them with the figures the government published (PIB releases): they match within 0.07% for four cycles, and an automated test checks this. District sums also equal state values in 100% of cases.

Q. Why did you exclude 2016-17?

Four states were missing and national totals came out about twice the published values, so the data from the API for that cycle is unreliable. Including it would make every cross-year answer wrong.

Q. Why do you need a crosswalk?

INGRES gives units new IDs in some cycles; only 14% of unit IDs survive from 2019-20 to 2021-22. Without linking, a question like 'which blocks moved from Critical to Over-Exploited' would silently miss most units.

Q. Can a user delete or change data through a question?

No. The connection is read-only at the database level and an authorizer callback rejects anything other than reading. This is enforced by SQLite itself, not by inspecting the query text, and is covered by 13 tests.

Q. What does the repair loop do and is it safe?

If the model's SQL fails or returns nothing, we send the error back and ask for a fix, at most twice. It only ever produces more read-only queries, so the safety layer still applies.

Q. Which component mattered most in the final system?

Retrieved examples: removing them drops test accuracy from 71.9% to 35.4%. Place hints add 4.4 points ($p = 0.019$) and the repair loop 1.6 points (it fixed 6 questions and broke none). Schema hints added 5.2 points over zero-shot.

Q. Isn't 71.9% inflated because test questions resemble training questions?

Partly, and we say so. Test question types also occur in training, with different places, years and wording, so retrieved examples are structurally similar. The keyword baseline has the same advantage and scores 32.6%; and hard_test, with noise and more complex questions, still reaches 57.7%. A leave-intents-out run is the planned check.

Q. Why SQLite and not PostgreSQL/MySQL?

The data is 22 MB and read-only; SQLite needs no server, ships as one file that teammates can copy, and its authorizer hook gives fine-grained read-only control.

4. Aryan: benchmark, evaluation, baselines

What was built

- **INGRES-Bench v1 (Phase 3):** 371 questions x 4 languages (English, Hindi, Hinglish, Tamil) = 1,484 instances, each with gold SQL and the gold result from the real database; 38 question types (intents) over 4 difficulty levels.
- **Quality control** with 7 automatic checks and a mutation test.
- **Execution-accuracy evaluator** with bootstrap confidence intervals and per-language/difficulty breakdowns.
- **Baselines (Phase 4):** keyword/template retrieval, BM25 and multilingual-embedding schema retrieval, and the local LLM (qwen3:8b) with a compact schema prompt.

How and why

Building the benchmark

How: each intent is an SQL template with slots (place, year, metric, category, threshold) plus at least 3 phrasings per language. A sampler draws real places and years from the database, the SQL is executed, and the item is kept only if the result is well-formed (non-empty, not NULL, at most 20 rows, exactly the intended unit). All four language versions are generated from the **same slots**, so they have identical SQL by construction. Places are mentioned with just enough context to be unambiguous (name, name + state, or name + district + state).

Why: no Text-to-SQL benchmark existed for Indian languages over Indian government data. Slot-aligned generation means a translation cannot change what the question asks. Difficulty levels: simple lookup (107 items), filtered aggregate (120), cross-year (80), multi-hop (64).

Preventing leakage

How: splits are by item, so all four languages of a question are in the same split (train 153 / dev 57 / test 96 / hard_test 65 items). Places are partitioned so test places are mostly unseen in training, and the last phrasing of every intent is held out of train/dev. hard_test adds noise to 201/260 instances (typos, real historical respellings, Latin names inside Hindi/Tamil, lowercase, numbers written as words). **Why:** otherwise a system could score well by memorising the Hindi version of a question it saw in English.

Quality control

How: 7 checks on all 1,484 instances (SQL executes, result reproduces, schema-valid, every SQL literal appears in the question, cross-language consistency, no split leakage, held-out phrasings): 0 failures. A mutation test corrupted 2,011 questions (replacing year, place or category); the slot checker caught 97.1%. 546 transliterated place names were back-transliterated: 543 pass, 3 flagged and judged correct. **Why:** generated benchmarks can contain silent errors; the mutation test proves the checker actually detects them.

Evaluation metric

How: `execution_match` runs predicted and gold SQL and compares results as multisets of rows (order matters only if the gold SQL has ORDER BY), numbers equal within relative tolerance 1e-6. Errors and refused SQL count as wrong. 95% confidence intervals by bootstrap; paired comparisons by McNemar's test. **Why:** many different SQL strings are correct, so comparing strings would under-count; comparing results is the standard (Spider/BIRD) way.

Baselines

Keyword/template (A): find the most similar training question (character n-gram TF-IDF), reuse its SQL, and re-fill years, categories, numbers, metric and place names by rules. EX 38.8% overall, 32.6% on test. **LLM (D):** qwen3:8b via Ollama, temperature 0. The full schema (~9k tokens) overflowed the 8k context, so we wrote a compact schema prompt plus explicit conventions (year labels, spellings, join path). EX 18.2%. **Schema retrieval:** BM25 recall@3 0.267 (collapses to 0.009 on Hindi and Tamil script), off-the-shelf multilingual embeddings 0.409.

Why baselines matter: they define what 'improvement' means. A surprising but honest finding is that the simple template baseline beats the 8B LLM, because the benchmark is template-based and the LLM makes systematic convention errors; the later components target exactly those errors.

Why a local model and not Gemini/ChatGPT: the Gemini free tier allowed 20 requests per day (logged), and one evaluation pass is 872 requests. A local model is free, private and reproducible (fixed seed, cached responses).

Main files to show

File	What to point at
`src/benchmark/intents.py`	INTENTS list: SQL template + phrasings in 4 languages for each question type
`src/benchmark/build.py`	Builder.sample (131): draws real slots; render (236): writes each language version
`scripts/qc_benchmark.py`	the 7 QC checks and the mutation test
`src/evaluation/execution.py`	execution_match (67), results_match (37): the metric
`src/evaluation/report.py`	bootstrap_ci (34): confidence intervals
`src/baselines/keyword.py`	KeywordBaseline.predict (162): retrieve and re-fill
`src/baselines/llm.py`	OllamaClient.generate (134): cached LLM call; compact_schema_prompt (176); CONVENTIONS (221)
`docs/INGRES_BENCH.md`, `docs/BASELINES.md`	benchmark protocol and baseline results

Viva questions and answers

Q. An AI helped write your questions. Is the benchmark valid?

The SQL and answers come from the real database, and all 4 language versions share the same slots, so the meaning cannot drift. QC found 0 failures and the checker catches 97% of deliberately corrupted questions. The honest limitation is that wording is template-like and no native speaker has verified the Hindi/Tamil yet; that check is planned.

Q. Why execution accuracy and not exact SQL match?

The same answer can be written as many different SQL queries (different aliases, join orders). Comparing the results is fairer and is the standard metric in Spider and BIRD.

Q. Why is your keyword baseline better than the LLM?

Because questions come from templates, copying the most similar training question's SQL works well. The 8B model, with no examples, makes convention errors (e.g. treating 'the 2023 assessment' as 2023-2024). It is a known limitation of template benchmarks, and it is why the pipeline adds retrieved examples and hints.

Q. How do you stop the model seeing test questions during training?

Splits are by item, test places are mostly unseen, and one phrasing per question type is held out. QC verifies that no SQL or question text appears in two splits.

Q. What is hard_test?

A harder split: more cross-year and multi-hop questions, ambiguous place names, and noise such as typos and Latin names inside Hindi/Tamil sentences. Baseline D drops from 18.0% (test) to 13.1% on it.

Q. Why Tamil as the Dravidian language?

Tamil Nadu is data-rich in INGRES, the script is different from both Devanagari and Latin (useful for the tokenizer study), and open-source support for Tamil is good.

Q. What does a 95% confidence interval mean here?

We resample the questions 2,000 times and recompute accuracy; 95% of those values fall in the interval. It shows how much the number could change with a different sample of questions.

5. Akshit: learned schema linking and entity resolution

What was built

- **Schema linker (Phase 5):** multilingual-e5-small (118M parameters) fine-tuned to rank the 106 database columns for a question in any of the 4 languages.
- **Geographic entity resolver (Phase 6):** maps any spelling or script of a place name to the exact INGRES entry among 9,558 places.
- **Place linker:** finds place names inside a full question and resolves them (used by the pipeline).

How and why

Schema linking

Why: the schema is in English ('stage_of_extraction_pct'); a Tamil question shares no words with it. BM25 scored 0.009 recall@3 on Hindi and Tamil. Off-the-shelf multilingual embeddings reached 0.409 overall but only 0.276 on Tamil. The LLM picking the wrong column was one of its most common errors.

How: a bi-encoder: question and column description are embedded separately and ranked by cosine similarity. Training uses contrastive learning (InfoNCE / multiple-negatives ranking loss): for each (question, correct column) pair the model is pushed to score the correct column above the others in the batch. The key idea is **hard negatives**: for each pair we add 3 columns that are easy to confuse: semantically close ones (recharge vs extractable resource; extraction vs stage of extraction) and **the same column at another administrative level** (unit vs district vs state table). 712 training pairs, 3 epochs, batch 16, lr 2e-5, 386 s on CPU.

Result: column recall@3 0.409 -> 0.896 (English 0.954, Hindi 0.913, Hinglish 0.872, Tamil 0.846); recall@1 0.221 -> 0.710. **Generalisation test:** retraining with 25% of question types removed and testing only on those gives 0.851 (3 seeds, 0.75-0.91), so it did not just memorise question types. In the LLM prompt, its top-5 columns raised end-to-end accuracy from 18.0% to 23.2% (p = 0.008).

Entity resolution

Why: 169 unit names are shared across different states, INGRES spellings are unusual ('TAMILNADU', Himachal district codes like 'KNG'), and users write names in Devanagari or Tamil, with typos or old names.

How: (1) a gazetteer of 9,558 places with aliases; (2) normalisation: Devanagari/Tamil are romanised, then 'phonetically folded' (aa->a, sh->s, w->v, doubled letters removed) and a consonant skeleton is kept, because vowels vary most across transliterations; (3) candidates by character n-gram TF-IDF (top 30); (4) a logistic-regression ranker over 12 features: string similarities, e5 embedding similarity, and **hierarchy context** (is the candidate's state or district mentioned elsewhere in the question? does it conflict?). It was trained only on synthetic noisy mentions (typos, transliteration variants, script changes) of 70% of places and tested on the other 30%.

Result (top-1): held-out synthetic noise 0.841 -> 0.885 with context; hand-written Hindi/Tamil names 0.674 -> 0.740; real historical INGRES respellings 0.623 -> 0.754; **ambiguous names 0.447 -> 0.973** with context. Weakest: hand-written Tamil (0.65), because Tamil script does not mark voicing or aspiration.

Why logistic regression and not a big model: 12 interpretable features, trains in minutes, and the learned weights can be explained (edit similarity +21.5, embedding similarity +10.6, state conflict -8.8).

Place linker

How: scan every 1-3 word span of the question, resolve it, keep confident (score >= 0.9) non-overlapping matches, then re-resolve with the detected state as context. Spans made only of common question words are skipped. **Bug found and fixed:** the first version treated frequent state names (Punjab, Karnataka) as common words (F1 0.56); a word is now skipped only if it is frequent and not tied to one place. Dev F1 0.852 (precision 0.906, recall 0.804).

Main files to show

File	What to point at
`src/schema_linking/train.py`	hard_negatives (31): the confusable columns; train (69): the contrastive loop (InfoNCE loss)
`src/schema_linking/retrieval.py`	EmbeddingRetriever (100): ranking; rank_metrics: recall@k, MRR
`src/entity_resolution/gazetteer.py`	fold (42): phonetic folding; build_gazetteer (72)
`src/entity_resolution/resolver.py`	features (57): the 12 features; resolve (86); fit (101): training the ranker
`src/pipeline/agent.py`	PlaceLinker.link (51): finding places in a question
`scripts/train_schema_linker.py, train_entity_resolver.py`	training + evaluation entry points
`docs/SCHEMA_LINKING.md, ENTITY_RESOLUTION.md`	methods and full results

Viva questions and answers

Q. What is a hard negative and why does it help?

A wrong answer that looks very similar to the right one. Random wrong columns are easy to reject and teach little. Training against 'extraction' vs 'stage of extraction', or the same column in the district table instead of the unit table, forces the model to learn the distinctions that matter.

Q. Didn't the model just memorise the question templates?

We tested that: we removed 25% of question types from training and evaluated only on them. Recall@3 was 0.851 versus 0.454 for the untrained model, so most of the gain transfers to unseen question types.

Q. What is recall@3?

Of the columns the correct SQL needs, the share that appear among the linker's top 3 suggestions.

Q. Why fine-tune a small model instead of using a large one?

It has to run for every question in milliseconds (3.8 ms per query) and train on a CPU in minutes. Fine-tuned, the small model reaches 0.896, which is enough to measurably improve the LLM.

Q. How do you tell apart two places with the same name?

Hierarchy context. If the question also mentions the state or district, candidates in that state get a strong boost and conflicting ones a strong penalty. Accuracy on ambiguous names goes from 0.45 to 0.97.

Q. Why is Tamil the hardest?

Tamil script uses one letter for sounds that are distinct in Hindi/English (k/g, t/d, aspirated or not), so a romanised Tamil name can differ a lot from the INGRES spelling.

Q. Why train on synthetic noise?

There is no labelled dataset of misspelled Indian place names. We generate realistic noise (typos, transliteration variants, script changes) for 70% of places and test on the unseen 30%, plus on real data: hand-written names and 445 real historical respellings from INGRES itself.

6. Waie: multilingual analysis, answer trust, user interface

What was built

- **Tokenization and code-mixing study (Phase 7)** with a placebo control, plus three mitigation experiments.
- **Numeric grounding verifier (Phase 8)** that checks every number in an answer against the SQL result.
- **Streamlit UI (Phase 10)**, tested end to end with screenshots; the mid-term report and slides.

How and why

Tokenization study

Why: a common explanation for weaker performance on Indian languages is that tokenizers split their words into many pieces. We tested whether that explains our accuracy gap.

How: for 5 tokenizers (Qwen3 = our LLM, XLM-R = our schema linker, mBERT, MuRIL, GPT-4o) and all 1,484 questions we measured **fertility** (tokens per word) and the share of words split into pieces. Code-mixing was measured with the Code-Mixing Index (CMI), using word-level language tags. Then we correlated fertility with accuracy, within the same question across its 4 languages, and with a **placebo**: the keyword baseline, which uses no subword tokenizer at all, so fertility cannot affect it.

Results: Qwen3 fertility English 2.04, Hindi 5.01, Hinglish 2.25, Tamil 8.96: a Tamil question costs ~111 tokens vs 28 in English. MuRIL (Indian-language model) is nearly flat (1.38-1.67). Hinglish CMI 33.0, 96% of Hinglish questions mixed. Fertility vs LLM accuracy: rho = -0.09; but the **placebo correlation is stronger (-0.18)**. Conclusion: the correlation reflects language and question difficulty, not fragmentation itself.

Mitigations: translate to English first: 16.4% vs 18.2% direct (no help; the 8B model mistranslated domain terms, e.g. 'ati-dohit' (over-exploited) became 'extremely undernourished'). Vocabulary adaptation (78 whole-word tokens added to the schema linker): Tamil fragmentation 0.63 -> 0.43, but recall unchanged (0.896 -> 0.897). What did help: schema-linker hints, +5.2 points (36 wins vs 16 losses, McNemar p = 0.008).

Numeric verifier

Why: the LLM writes the final sentence and could invent or mis-scale a number; for official water data a fluent wrong number is worse than no answer.

How: all numbers in the answer are extracted after normalising Devanagari and Tamil digits, Indian grouping (1,23,456) and lakh/crore words. Each must match a value in the SQL result, allowing for the precision it is written with and unit conversions (hectare-metres <-> BCM, fraction <-> percent), the row count, list positions and numbers in the question. Years are exempt. If any number fails, the answer falls back to a plain rendering of the table.

Results (384 test questions): LLM answers with an unsupported number: 0.83% (2/242); both checked by hand and both real errors ('8 blocks' for a 7-row result; '82.69%' for a true 8.27%). Planted errors caught: 98.1% (211/215; the 4 misses became 1900-2099 values, treated as years). False alarms: 0%. After the fallback: 0% unsupported numbers. The evaluation also found two bugs in the verifier (SQL-expression column headers; lakh precision), fixed with tests.

User interface

How: a Streamlit page: question box with examples in all 4 languages; shows the answer, a warning if the verifier replaced the model's wording, the SQL, the result table, and a 'How it got there' panel (schema hints, resolved places, repair attempts, latency). **Why:** showing the SQL and table lets a user check the answer instead of trusting it blindly. Tested end to end: a Hindi question about Ludhiana returned the correct SQL and a Hindi answer (240,136.75 ham). Screenshots: `reports/screenshots/`.

Main files to show

File	What to point at
`src/tokenization/metrics.py`	tokenizer_stats (34): fertility; LanguageTagger.cmi (91): code-mixing index

File	What to point at
`scripts/run_tokenization_study.py`	within-item correlation and the placebo analysis
`src/tokenization/mitigation.py`	translate (22); select_new_tokens (29); adapt_vocabulary (42)
`src/grounding/verifier.py`	numbers (24): Indic digits + lakh; verify (59); grounded_answer (98): fallback
`scripts/run_grounding.py`	the perturbation test and false-alarm checks
`app/streamlit_app.py`	the UI
`docs/TOKENIZATION.md`, `docs/GROUNDING.md`	methods and full results

Viva questions and answers

Q. What is fertility and why measure it?

The average number of tokens a tokenizer uses per word. High fertility means words are chopped into many pieces, which costs context, time and possibly accuracy. Qwen3 uses ~9 tokens per Tamil word vs 2 per English word.

Q. What is a placebo control here?

A variable that should not be affected. The keyword baseline uses no subword tokenizer, so tokenizer fertility cannot cause its errors. It still correlated with fertility, even more strongly, which shows the original correlation is caused by something else: harder languages and items.

Q. So does tokenization not matter at all?

It matters for cost and speed (Tamil prompts are ~4x longer). For accuracy, in our system, it was not the bottleneck: reducing fragmentation did not raise accuracy, while better schema linking did.

Q. Why did translating to English first make things worse?

The 8B model translated technical terms into everyday meanings, e.g. 'critical' became 'severe' and 'stage of extraction' became 'attendance level', and occasionally dropped place names. The original terms were more useful to the SQL model.

Q. What is code-mixing and how did you measure it?

Using two languages in one sentence, like Hinglish. Each word is tagged by language; the Code-Mixing Index is 0 for a single language and rises as languages mix. Hinglish averaged 33.

Q. What does your verifier not catch?

It checks that a number exists in the result, not that it is attached to the right place, and it skips years and numbers written as words. Small counts can match a list position by chance.

Q. Why fall back to a table instead of asking the LLM again?

The table rendering is correct by construction and costs nothing; asking again could produce another error.

7. Questions any member may be asked

Q. Explain the project in two sentences.

People ask about groundwater in Hindi, Hinglish or Tamil, but the official portal is an English click-through map. We built the database, a 4-language benchmark and a pipeline that turns such questions into safe SQL with verified answers, and measured which components actually help.

Q. What is novel?

A Text-to-SQL benchmark over Indian government data in 4 languages; a schema linker trained with domain and administrative-level hard negatives; a place resolver using the administrative hierarchy; a placebo-controlled tokenization study; and a numeric verifier for Indian number formats.

Q. Why not just use ChatGPT?

Cost and quota (one evaluation is 872 calls; the free Gemini tier allows 20 per day), reproducibility (a local model with temperature 0 gives identical results), privacy, and a general chatbot does not know INGRES spellings, year conventions or units. A paid frontier-model comparison is planned as future work.

Q. What hardware did you use?

An NVIDIA RTX 5060 (8 GB) ran the LLM (qwen3:8b, 4-bit) through Ollama: about 3,700 calls, ~2.1 GPU-hours, ~2.5 s per question. Our own models (118M-parameter encoder, logistic regression) were trained on the CPU (Ryzen 7 9700X) in about 6 minutes each, because they are small and the installed PyTorch build is CPU-only.

Q. What is the biggest weakness?

Most of the gain comes from retrieved examples, and test question types also appear in training, so accuracy on truly new question types is probably below 71.9%. The Hindi/Tamil questions have not been verified by native speakers, and Tamil place names remain the biggest single error source.

Q. How do you make sure your numbers are not made up?

Every reported number is written by a script into a results file in the repository; responses from the LLM are cached, so anyone can rerun and get the same numbers; 61 automated tests cover the data, safety, metrics and verifier.

Q. What would you do next?

A leave-intents-out evaluation; better Tamil place resolution (41% of errors are place misses); a deterministic year normaliser (13% of errors); native-speaker review of a sample; a frontier-model comparison; re-timing training on the GPU.

8. Glossary

Term	Meaning
EX (execution accuracy)	share of questions whose predicted SQL returns the same result as the gold SQL
Recall@3	share of the needed columns that appear in the schema linker's top 3
Fertility	subword tokens per word
CMI	Code-Mixing Index: 0 = one language; higher = more mixing
ham / BCM	hectare-metre / billion cubic metres; 1 BCM = 100,000 ham
Stage of extraction	annual extraction / extractable resource x 100; above 100% = Over-Exploited
GWRA / GEC-2015	Ground Water Resource Assessment cycle / the official methodology
McNemar test	paired significance test counting questions one system gets right and the other wrong
Contrastive learning (InfoNCE)	training that pulls matching pairs together and pushes non-matching apart
Ablation	switching one component off to measure its contribution